

ZEAL IT CONSULTANTS

Architecture is Like and Onion



What is Layered Architecture?

Layered architecture is a design pattern that structures an application into distinct, horizontal layers, each with a well-defined role and responsibility.

Examples:

- Presentation Layer
- Application Layer
- Domain Layer
- Persistence Layer



Why is layered architecture important?

Layered architecture provides a number of benefits:

- Separation of Concerns
- Scalability
- Testability
- Reusability
- Parallel Development
- Maintainability and Modularity
- Interchangeability and Technology Agility
- Enhanced Security



Why is layered architecture important?

Separation of Concerns

Each layer has a dedicated responsibility, such as handling data, implementing business rules, or presenting information to the user. This separation makes the application easier to understand and maintain since changes in one area (like the UI) don't necessarily impact others (like the business logic).

- Models
 - Persistence
 - Domain
 - Presentation
- Services



Why is layered architecture important?

Scalability

With clearly defined layers, parts of the application can scale independently. For instance, the data layer can be optimized or scaled separately from the application layer, improving performance without needing a full system overhaul.

- Caching
- Selection of Technology



Why is layered architecture important?

Testability and Parallel Development

Layers allow for isolated testing and Development. You can mock or stub out other layers when testing one layer, making it easier to verify functionality without setting up the entire system. For example, you can test business logic without needing a database connection.

- UI
- Backend
- Presentation



Why is layered architecture important?

Reusability

Well-defined layers promote reusability. Components within a layer can be reused in different applications, such as the data access layer or business services, which can serve other apps with similar needs.

- Email Services
- Authentication/Authorization



Why is layered architecture important?

Maintainability and Modularity

Layered architecture encourages modular code that is easier to maintain. If a layer needs updates, you can focus on that layer without having to refactor the entire application, minimizing the risk of introducing bugs.



Why is layered architecture important?

Interchangeability and Technology Agility

Layered architecture can make it easier to swap out technology in a specific layer.

For example, if you need to change the data storage technology, you can make changes primarily in the data layer without altering the rest of the application.



Why is layered architecture important?

Enhanced Security

Since each layer has a defined role, you can apply security measures tailored to that layer's needs, isolating sensitive operations.

- Data Layer
 - Security / Encryption
- Presentation Layer
 - View Models to expose only necessary information



Who cares?

Although architecture is implemented primarily by architects and developers, layered architecture impacts all roles involved in the software development process.



Who cares?

Technical Roles

Software Architects

Responsible for defining the overarching structure of a system. They ensure that the architecture supports the desired level of maintainability, scalability, and separation of concerns. Layered architecture helps them plan how different parts of the system interact and remain cohesive.

Developers

Developers are the ones implementing the code within the constraints of the architecture. Understanding layered architecture helps them write better, more modular code that fits within the overall design, making it easier to maintain and extend.

Tech/Team Leads

Responsible for ensuring the technical direction of the project is followed and that best practices are maintained across the team. A layered architecture helps them guide the team and enforce standards.

QA Engineers

A layered architecture makes testing easier because different layers can be tested independently. QA teams benefit from understanding this structure to create more effective test cases and strategies.



Who cares?

Non-Technical Roles

Project Managers

While not involved in the code, project managers need to understand how architecture affects timelines and the ability to meet project milestones. A well-defined layered architecture can speed up development by reducing complexity and facilitating parallel development.

DevOps Engineers

DevOps teams are involved in deploying and maintaining applications. Understanding the architecture helps them set up CI/CD pipelines that cater to the needs of the system, such as deploying changes to individual layers or scaling specific parts of the application.

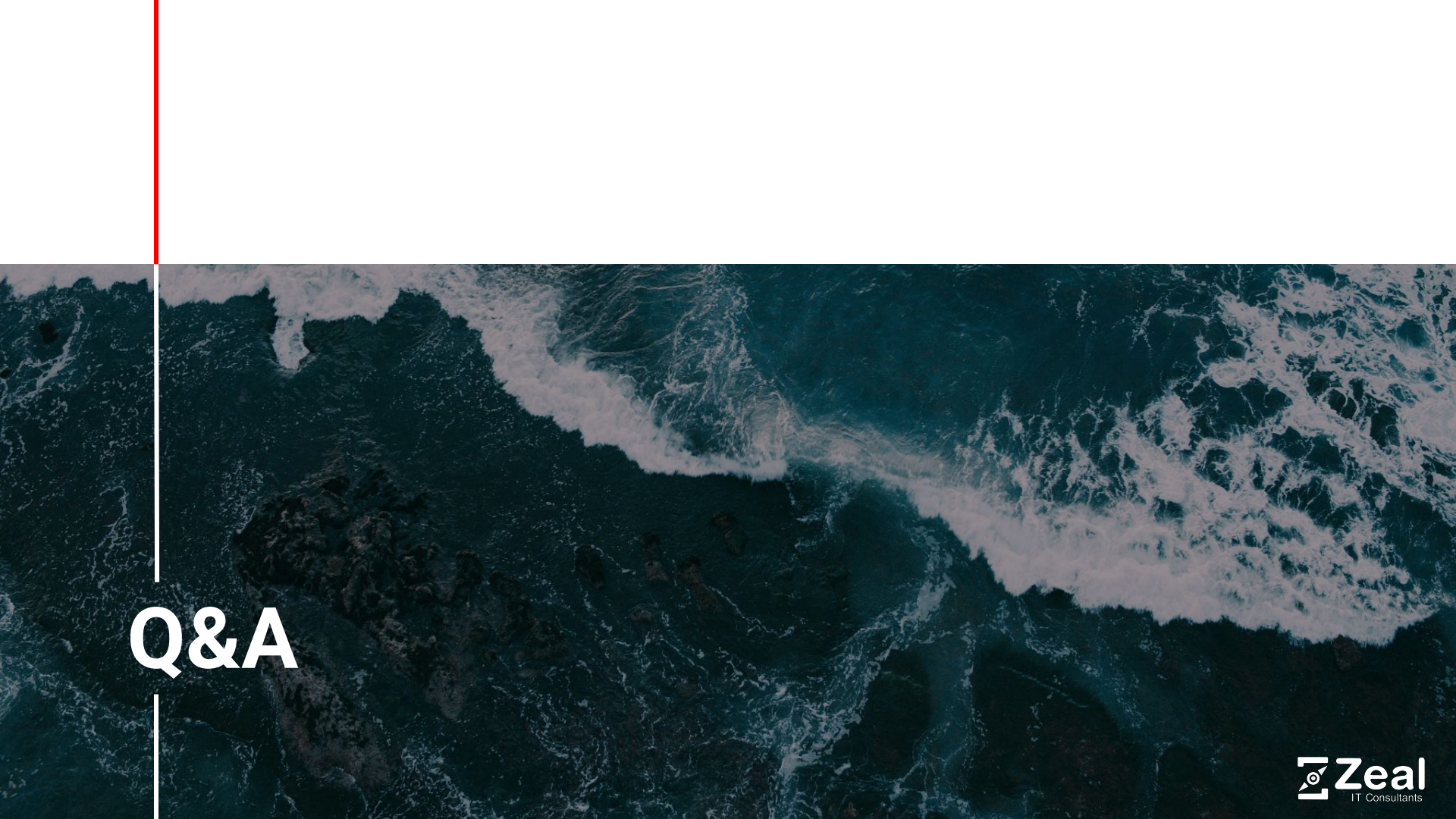
Product Owners

They need to understand how the architecture might affect the implementation of business requirements and future enhancements. Layered architecture often helps explain the limitations or capabilities of the system when making decisions.

Security Analysts

A layered architecture provides clear points where security can be implemented and enforced. Analysts can use the structure to identify potential vulnerabilities and ensure that the proper security measures are in place at each layer.





Q&A

ZEAL IT CONSULTANTS

Thanks!

